

Simulation of the depth of field

Dokumentáció

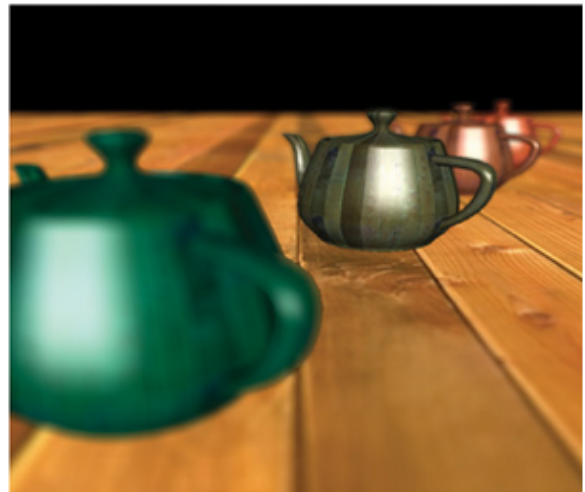
Használat

A Project NewProject néven van elmentve a mappában, el kell indítani paraméterek nélkül és a projektben futás közben lehet kiválasztani a képeket

Feladat:

A [summed-area table \(SAT\)](#), also called an integral image is a 2D table generated from an input image in which each entry in the table stores the sum of all pixels between the entry location and the lower-left corner of the input image. The power of the summed-area table comes from the fact that it can be used to perform filters of different widths at every pixel in the image in constant time per pixel. Generating a box-filtered pixel using a summed-area table requires sampling the summed-area table at the four corners of a rectangular filter region, s_{ur} , s_{ul} , s_{lr} , s_{ll} .

$$s_{filter} = \frac{s_{ur} - s_{ul} - s_{lr} + s_{ll}}{w \times h},$$



where w and h are the width and height of the filter kernel, and s_{ur} is the upper-right corner sample, s_{ll} is the lower-left corner sample, and so on. We can use this technique for variable-width filtering, by varying the locations of the four samples we use to compute each filtered output pixel.

Tasks:

- Input:
 - a. Read a pair of input RGB and depth images **(5%)**
 - b. OR use real-time OpenGL/DX/Vulkan rendering of a scene with color and depth rendered to frame buffers directly **(15%)**.
- Use the Scan parallel pattern to implement the [Summed-Area Table](#). **(20%)**
- Use the variable-width box filter to compute the smoothed image. **(20%)**
- Save the output image to a file. **(5%)**
- Use real-time *display* **(20%)** and *change* the parameters for focus in real-time. **(20%)**

Feladat értelmezése:

A feladat azt várja, hogy írjunk egy programot, amiben kapunk kettő képet, egy színes, módosítandó képet, és egy fekete-fehéret, ahol a színek a homályosítás erősségét jelzi, ezt a homályosítást majd alkalmazzuk a színes képre és adjuk vissza

Ezt a kész képet el lehessen menteni, és a homályosítás paramétereit valós időben lehessen változtatni

Megvalósítás:

A programban, először egy Blleloch scannel létrehozuk az SAT (Summed Area Table)-t, majd azon alkalmazva a box filtert létrehozuk a kész képet. Végül elmentjük egy fájlba, amit odaadunk a felhasználónak

Blleloch scan: Egy párhuzamos összegzési módszer, ami gyors SAT építést tesz lehetővé

OpenCl és OpenGL segítségével valós időben lehet változtatni a homályosítás paramétereit, valamint, hogy milyen formátumban szeretnénk elmenteni a kész képünket

A Program:

Kernelek:

```
__kernel void simple_sat_horizontal(__global float4* input, __global float4* horizontal,
                                   int width, int height) {
    int y = get_global_id(0);
    if (y >= height) return;

    float4 sum = (float4)(0.0f);
    for (int x = 0; x < width; x++) {
        int idx = y * width + x;
        sum += input[idx];
        horizontal[idx] = sum;
    }
}

//Kernel for vertical SAT creation
__kernel void simple_sat_vertical(__global float4* horizontal, __global float4* sat,
                                  int width, int height) {
    int x = get_global_id(0);
    if (x >= width) return;

    float4 sum = (float4)(0.0f);
    for (int y = 0; y < height; y++) {
        int idx = y * width + x;
        sum += horizontal[idx];
        sat[idx] = sum; // Final SAT value
    }
}
```

Kernelek az SAT létrehozásához, az egyik a vízszintes a másik a függőleges sorokat készíti

```

__kernel void apply_depth_blur_simple(__global float4* col, __global float* dep,
                                     __global float4* sat, __write_only image2d_t out,
                                     int w, int h, float scale, int invertDepth) {
    int x = get_global_id(0), y = get_global_id(1);
    if (x >= w || y >= h) return;

    float depth_val = dep[y * w + x];
    if (invertDepth != 0) depth_val = 1.0f - depth_val;

    int r = max(0, (int)(depth_val * scale + 0.5f));

    if (r == 0) {
        float4 original = col[y * w + x];
        write_imagef(out, (int2)(x, y), (float4)(clamp(original.xyz, 0.0f, 1.0f), 1.0f));
        return;
    }

    // Rectangle bounds (inclusive)
    int left = max(0, x - r);
    int right = min(w - 1, x + r);
    int top = max(0, y - r);
    int bottom = min(h - 1, y + r);

    // SAT indices (need to handle edges)
    int br_idx = bottom * w + right;
    float4 br = sat[br_idx];

    // For top-1, left-1 coordinates
    int tr_idx = (top > 0) ? (top - 1) * w + right : -1;
    int bl_idx = (left > 0) ? bottom * w + (left - 1) : -1;
    int tl_idx = (top > 0 && left > 0) ? (top - 1) * w + (left - 1) : -1;

    float4 tr = (tr_idx >= 0) ? sat[tr_idx] : (float4)(0.0f);
    float4 bl = (bl_idx >= 0) ? sat[bl_idx] : (float4)(0.0f);
    float4 tl = (tl_idx >= 0) ? sat[tl_idx] : (float4)(0.0f);

    float4 sum = br - tr - bl + tl;

    float area = (float)((right - left + 1) * (bottom - top + 1));
    float4 average = sum / max(area, 1.0f);

    write_imagef(out, (int2)(x, y), (float4)(clamp(average.xyz, 0.0f, 1.0f), 1.0f));
}

```

Kernel amely homályosítást végez

```

__kernel void simple_box_blur(__global float4* col, __global float* dep,
                             __write_only image2d_t out, int w, int h,
                             float scale, int invertDepth) {
    int x = get_global_id(0), y = get_global_id(1);
    if (x >= w || y >= h) return;

    float depth_val = dep[y * w + x];
    if (invertDepth != 0) depth_val = 1.0f - depth_val;

    int r = max(0, (int)(depth_val * scale + 0.5f));

    if (r == 0) {
        float4 original = col[y * w + x];
        write_imagef(out, (int2)(x, y), (float4)(clamp(original.xyz, 0.0f, 1.0f), 1.0f));
        return;
    }

    float4 sum = (float4)(0.0f);
    int count = 0;

    for (int dy = -r; dy <= r; dy++) {
        int ny = y + dy;
        if (ny < 0 || ny >= h) continue;

        for (int dx = -r; dx <= r; dx++) {
            int nx = x + dx;
            if (nx < 0 || nx >= w) continue;

            sum += col[ny * w + nx];
            count++;
        }
    }

    if (count > 0) {
        float4 average = sum / (float)count;
        write_imagef(out, (int2)(x, y), (float4)(clamp(average.xyz, 0.0f, 1.0f), 1.0f));
    } else {
        float4 original = col[y * w + x];
        write_imagef(out, (int2)(x, y), (float4)(clamp(original.xyz, 0.0f, 1.0f), 1.0f));
    }
}

```

Egyszerű box blur összehasonlítás képpen, valamint alternatív használatra

```
__kernel void visualize_depth(__global float* depth, __write_only image2d_t out,
                             int w, int h, int invertDepth) {
    int x = get_global_id(0), y = get_global_id(1);
    if (x >= w || y >= h) return;

    float d = depth[y * w + x];
    if (invertDepth != 0) d = 1.0f - d;

    write_imagef(out, (int2)(x, y), (float4)(d, d, d, 1.0f));
}
```

Vizualizáló kernel, kirajzolja kész képet

A C++ Program:

Segédfüggvények:

```
static void checkErr(cl_int err, const char* name) {
    if (err != CL_SUCCESS) {
        std::cerr << "OpenCL error (" << name << "): " << err << std::endl;
        std::exit(EXIT_FAILURE);
    }
}
```

Megnézi, hogy történt-e hiba, minden lépés után használandó, ellenőrzésképpen

```
static std::string loadKernelSource(const char* file) {
    std::ifstream f(file);
    if (!f.is_open()) {
        std::cerr << "Failed to open kernel file: " << file << std::endl;
        return "";
    }
    std::stringstream ss;
    ss << f.rdbuf();
    return ss.str();
}
```

Kernel beolvasó, hogy a c++ kód megértse az előbb mutatott kerneleket

```
float clamp(float value, float min, float max) {
    if (value < min) return min;
    if (value > max) return max;
    return value;
}
```

Egyszerű clamp függvény

```
std::string OpenFileDialog(const char* filter = "Image Files\\0*.jpg;*.jpeg;*.png;*.bmp;*.tga;\\0All Files\\0*.*\\0") {
    OPENFILENAMEA ofn;
    char fileName[MAX_PATH] = "";

    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = NULL;
    ofn.lpstrFilter = filter;
    ofn.lpstrFile = fileName;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_FILEMUSTEXIST | OFN_PATHMUSTEXIST;
    ofn.lpstrDefExt = "";

    if (GetOpenFileNameA(&ofn)) {
        return std::string(fileName);
    }
    return "";
}
```

Ez fogja kiadni a kép választáshoz a tallózás opciót

```

std::string SaveFileDialog(const char* filter = "PNG Image\0*.png\0JPEG Image\0*.jpg;*.jpeg\0BMP Image\0*.bmp\0All Files\0*.*\0") {
    OPENFILENAMEA ofn;
    char fileName[MAX_PATH] = "output.png";

    ZeroMemory(&ofn, sizeof(ofn));
    ofn.lStructSize = sizeof(ofn);
    ofn.hwndOwner = NULL;
    ofn.lpstrFilter = filter;
    ofn.lpstrFile = fileName;
    ofn.nMaxFile = MAX_PATH;
    ofn.Flags = OFN_OVERWRITEPROMPT;
    ofn.lpstrDefExt = "png";

    if (GetSaveFileNameA(&ofn)) {
        return std::string(fileName);
    }

    return "";
}

```

A fájl mentéséhez fog segíteni, fogja adni az opciókat, pl. formátum

```

std::vector<float> ReadTexturePixelsFloat(GLuint texture, int width, int height) {
    std::vector<float> pixels(width * height * 4);

    glBindTexture(GL_TEXTURE_2D, texture);

    glGetTexImage(GL_TEXTURE_2D, 0, GL_RGBA, GL_FLOAT, pixels.data());

    glBindTexture(GL_TEXTURE_2D, 0);

    return pixels;
}

```

Képek beolvasásában segít, hogy érthető legyen a c++ kód számára egy kép és annak a színei

Jelenleg a program csak a png, jpg(jpeg), bmp formátumokat támogatja

```

bool SaveImage(const std::string& filename, const std::vector<unsigned char>& pixels, int width, int height, int channels = 4) {
    std::string extension = filename.substr(filename.find_last_of(".") + 1);
    std::transform(extension.begin(), extension.end(), extension.begin(), ::tolower);

    int result = 0;

    if (extension == "png") {
        result = stbi_write_png(filename.c_str(), width, height, channels, pixels.data(), width * channels);
    }
    else if (extension == "jpg" || extension == "jpeg") {
        result = stbi_write_jpg(filename.c_str(), width, height, channels, pixels.data(), 90);
    }
    else if (extension == "bmp") {
        result = stbi_write_bmp(filename.c_str(), width, height, channels, pixels.data());
    }
    else {
        std::cerr << "Unsupported file format: " << extension << std::endl;
        return false;
    }

    if (result) {
        std::cout << "Saved image to: " << filename << std::endl;
        return true;
    }
    else {
        std::cerr << "Failed to save image: " << filename << std::endl;
        return false;
    }
}

```

A mentésért felelő függvény, 3 formátumot támogat a program, png, jpg (jpeg) és bmp-t

Főprogram:

```

// Initialize GLFW
if (!glfwInit()) {
    std::cerr << "Failed to initialize GLFW" << std::endl;
    return -1;
}

// Create window
const int w = 512, h = 512;
GLFWwindow* window = glfwCreateWindow(w, h, "Depth Blur - 512x512", nullptr, nullptr);
if (!window) {
    std::cerr << "Failed to create GLFW window" << std::endl;
    glfwTerminate();
    return -1;
}

glfwMakeContextCurrent(window);
glfwSwapInterval(1);

// Initialize GLAD
if (!gladLoadGLLoader((GLADloadproc)glfwGetProcAddress)) {
    std::cerr << "Failed to initialize GLAD" << std::endl;
    return -1;
}

// Initialize OpenCL
cl_platform_id platform;
cl_device_id device;
cl_int err;

err = clGetPlatformIDs(1, &platform, nullptr);
checkErr(err, "clGetPlatformIDs");

err = clGetDeviceIDs(platform, CL_DEVICE_TYPE_GPU, 1, &device, nullptr);
if (err != CL_SUCCESS) {
    std::cerr << "No GPU found, trying CPU..." << std::endl;
    err = clGetDeviceIDs(platform, CL_DEVICE_TYPE_CPU, 1, &device, nullptr);
    checkErr(err, "clGetDeviceIDs");
}

// Create OpenCL context with GL sharing
cl_context_properties props[] = {
    CL_GL_CONTEXT_KHR, (cl_context_properties)glfwGetWGLContext(window),
    CL_WGL_HDC_KHR, (cl_context_properties)GetDC(glfwGetWin32Window(window)),
    CL_CONTEXT_PLATFORM, (cl_context_properties)platform, 0
};

cl_context context = clCreateContext(props, 1, &device, nullptr, nullptr, &err);
checkErr(err, "clCreateContext");

cl_command_queue queue = clCreateCommandQueue(context, device, 0, &err);
checkErr(err, "clCreateCommandQueue");

```

GLFW, GLAD, OpenCL és az OpenCL kontextusa inicializálása OpenGL segítségével, ez a kontextusmegosztás lehetővé teszi a textúrák közvetlen átadását az két API között, ami gyorsítja a renderelést


```

// Load and compile kernels
std::string src = loadKernelSource("kernels.cl");
if (src.empty()) {
    std::cerr << "Failed to load kernel source" << std::endl;
    return -1;
}

const char* srcPtr = src.c_str();
size_t srcLen = src.size();
cl_program program = clCreateProgramWithSource(context, 1, &srcPtr, &srcLen, &err);
checkErr(err, "clCreateProgramWithSource");

err = clBuildProgram(program, 1, &device, nullptr, nullptr, nullptr);
if (err != CL_SUCCESS) {
    size_t log_size;
    clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG, 0, nullptr, &log_size);
    std::vector<char> log(log_size);
    clGetProgramBuildInfo(program, device, CL_PROGRAM_BUILD_LOG, log_size, log.data(), nullptr);
    std::cerr << "Build log:\n" << log.data() << std::endl;
    return -1;
}

// Create kernels
cl_kernel sat_horizontal = clCreateKernel(program, "simple_sat_horizontal", &err);
checkErr(err, "clCreateKernel sat_horizontal");

cl_kernel sat_vertical = clCreateKernel(program, "simple_sat_vertical", &err);
checkErr(err, "clCreateKernel sat_vertical");

cl_kernel blurKernel = clCreateKernel(program, "apply_depth_blur_simple", &err);
checkErr(err, "clCreateKernel blurKernel");

cl_kernel simpleBlurKernel = clCreateKernel(program, "simple_box_blur", &err);
checkErr(err, "clCreateKernel simpleBlurKernel");

cl_kernel visualizeDepthKernel = clCreateKernel(program, "visualize_depth", &err);
checkErr(err, "clCreateKernel visualizeDepthKernel");

```

Kernelek betöltése és létrehozása, hogy tudjuk őket használni a c++ kódunkban

```

// Buffer sizes
const size_t pixels = w * h;
const size_t colorBufferSize = sizeof(float) * 4 * pixels;
const size_t depthBufferSize = sizeof(float) * pixels;

// Create OpenCL buffers
cl_mem colorBuffer = clCreateBuffer(context, CL_MEM_READ_ONLY, colorBufferSize, nullptr, &err);
checkErr(err, "clCreateBuffer colorBuffer");

cl_mem depthBuffer = clCreateBuffer(context, CL_MEM_READ_ONLY, depthBufferSize, nullptr, &err);
checkErr(err, "clCreateBuffer depthBuffer");

cl_mem satBuffer = clCreateBuffer(context, CL_MEM_READ_WRITE, colorBufferSize, nullptr, &err);
checkErr(err, "clCreateBuffer satBuffer");

cl_mem tempBuffer = clCreateBuffer(context, CL_MEM_READ_WRITE, colorBufferSize, nullptr, &err);
checkErr(err, "clCreateBuffer tempBuffer");

```

Bufferek létrehozása, amiket fogunk majd használni

```

// Create OpenGL texture for main output (32-bit float)
GLuint outputTexture;
glGenTextures(1, &outputTexture);
glBindTexture(GL_TEXTURE_2D, outputTexture);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA32F, w, h, 0, GL_RGBA, GL_FLOAT, nullptr);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

// Create OpenCL image from OpenGL texture
cl_mem outputBuffer = clCreateFromGLTexture(context, CL_MEM_WRITE_ONLY, GL_TEXTURE_2D, 0, outputTexture, &err);
checkErr(err, "clCreateFromGLTexture outputBuffer");

// Create depth visualization texture
GLuint depthTexture;
glGenTextures(1, &depthTexture);
glBindTexture(GL_TEXTURE_2D, depthTexture);
glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA32F, w, h, 0, GL_RGBA, GL_FLOAT, nullptr);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

cl_mem depthVisBuffer = clCreateFromGLTexture(context, CL_MEM_WRITE_ONLY, GL_TEXTURE_2D, 0, depthTexture, &err);
checkErr(err, "clCreateFromGLTexture depthVisBuffer");

```

OpenGL és OpenCL textúrák létrehozása, azért, hogy a képeket valós időben tudjuk majd kedvünk szerint változtatgatni

```

// Load or create test data
std::vector<float> colorData(pixels * 4, 0.0f);
std::vector<float> depthData(pixels, 0.0f);

// File paths
std::string colorImagePath = "image.jpg";
std::string depthImagePath = "depth.jpg";

// Try to load images
int iw = 0, ih = 0, ic = 0;
float* imgData = nullptr;
float* depData = nullptr;

auto loadImages = [&]() -> bool {
    // Free previous data
    if (imgData) stbi_image_free(imgData);
    if (depData) stbi_image_free(depData);
    imgData = nullptr;
    depData = nullptr;

    // Try to load color image
    imgData = stbi_loadf(colorImagePath.c_str(), &iw, &ih, &ic, 4);
    if (!imgData) {
        std::cout << "Failed to load color image: " << colorImagePath << std::endl;
        return false;
    }

    // Try to load depth image
    depData = stbi_loadf(depthImagePath.c_str(), &iw, &ih, &ic, 1);
    if (!depData) {
        std::cout << "Failed to load depth image: " << depthImagePath << std::endl;
        stbi_image_free(imgData);
        imgData = nullptr;
        return false;
    }

    if (iw != w || ih != h) {
        std::cout << "Image size mismatch! Expected " << w << "x" << h
            << ", got " << iw << "x" << ih << std::endl;
        stbi_image_free(imgData);
        stbi_image_free(depData);
        imgData = nullptr;
        depData = nullptr;
        return false;
    }

    // Copy data to vectors
    memcpy(colorData.data(), imgData, colorBufferSize);
    memcpy(depthData.data(), depData, depthBufferSize);

    // Write to OpenCL buffers
    err = clEnqueueWriteBuffer(queue, colorBuffer, CL_TRUE, 0, colorBufferSize, colorData.data(), 0, nullptr, nullptr);
    checkErr(err, "clEnqueueWriteBuffer colorBuffer");

    err = clEnqueueWriteBuffer(queue, depthBuffer, CL_TRUE, 0, depthBufferSize, depthData.data(), 0, nullptr, nullptr);
    checkErr(err, "clEnqueueWriteBuffer depthBuffer");

    std::cout << "Loaded images successfully: " << colorImagePath << ", " << depthImagePath << std::endl;
    return true;
};

```

A képek betöltése, majd átadása a vectorokban az OpenCL buffereknek

FONTOS! A program jelenleg csak az 520 x 520-as formátumú képeket támogatja

```

auto createTestPattern = [&]() {
    std::cout << "Using test pattern" << std::endl;
    // Create checkerboard pattern
    for (int y = 0; y < h; y++) {
        for (int x = 0; x < w; x++) {
            int idx = y * w + x;
            int color_idx = idx * 4;

            // Checkerboard pattern
            bool isWhite = ((x / 32) + (y / 32)) % 2 == 0;
            float val = isWhite ? 1.0f : 0.3f;

            colorData[color_idx] = val;
            colorData[color_idx + 1] = val;
            colorData[color_idx + 2] = val;
            colorData[color_idx + 3] = 1.0f;

            // Depth: circle in center
            int cx = w / 2;
            int cy = h / 2;
            float dist = sqrtf(powf(x - cx, 2) + powf(y - cy, 2));
            depthData[idx] = (dist < 100.0f) ? 1.0f : 0.0f;
        }
    }

    // Write to OpenCL buffers
    err = clEnqueueWriteBuffer(queue, colorBuffer, CL_TRUE, 0, colorBufferSize, colorData.data(), 0, nullptr, nullptr);
    checkErr(err, "clEnqueueWriteBuffer colorBuffer");

    err = clEnqueueWriteBuffer(queue, depthBuffer, CL_TRUE, 0, depthBufferSize, depthData.data(), 0, nullptr, nullptr);
    checkErr(err, "clEnqueueWriteBuffer depthBuffer");
};

// Try to load images initially
if (!loadImages()) {
    createTestPattern();
}

```

Ha nem adunk meg képet, akkor egy teszteléshez használt sakktábla szerű képet fogunk látni, melynek a közepén egy kört lehet homályosítani kedv szerint

```

// Initialize ImGui
ImGui::CreateContext();
ImGuiIO& io = ImGui::GetIO();
io.ConfigFlags |= ImGuiConfigFlags_NavEnableKeyboard;

ImGui_ImplGlfw_InitForOpenGL(window, true);
ImGui_ImplOpenGL3_Init("#version 330");

// Application state
float blurRadius = 10.0f;
bool useSATMethod = true;
bool showDepthMap = false;
bool invertDepth = false;
float depthMultiplier = 1.0f;

// File loading state
bool reloadRequested = false;
bool loadColorRequested = false;
bool loadDepthRequested = false;
bool useTestPatternRequested = false;
bool saveOutputRequested = false;

// Display strings
char colorPathBuffer[256] = "image.jpg";
char depthPathBuffer[256] = "depth.jpg";

```

ImGui segítségével fogjuk létrehozni a UI-t, valamint a képek betöltését

Majd segéd elemek létrehozása, amik később lesznek használva

```

while (!glfwWindowShouldClose(window)) {
    glfwPollEvents();

    // Start ImGui frame
    ImGui_ImplOpenGL3_NewFrame();
    ImGui_ImplGlfw_NewFrame();
    ImGui::NewFrame();

    // Settings window
    ImGui::Begin("Settings");

    // File loading section
    ImGui::SeparatorText("File Loading");

    ImGui::InputText("Color Image", colorPathBuffer, sizeof(colorPathBuffer));
    ImGui::SameLine();
    if (ImGui::Button("Browse##Color")) {
        loadColorRequested = true;
    }

    ImGui::InputText("Depth Image", depthPathBuffer, sizeof(depthPathBuffer));
    ImGui::SameLine();
    if (ImGui::Button("Browse##Depth")) {
        loadDepthRequested = true;
    }

    ImGui::Spacing();
    if (ImGui::Button("Load Images")) {
        colorImagePath = colorPathBuffer;
        depthImagePath = depthPathBuffer;
        reloadRequested = true;
    }

    ImGui::SameLine();
    if (ImGui::Button("Use Test Pattern")) {
        useTestPatternRequested = true;
    }

    // Save output section
    ImGui::SeparatorText("Save Output");
    if (ImGui::Button("Save Output Image (PNG/JPG/BMP)")) {
        saveOutputRequested = true;
    }

    // Blur settings
    ImGui::SeparatorText("Blur Settings");
    ImGui::SliderFloat("Blur Radius", &blurRadius, 0.0f, 50.0f);
    ImGui::Checkbox("Use SAT Method (Fast)", &useSATMethod);
    ImGui::Checkbox("Show Depth Map", &showDepthMap);
    ImGui::Checkbox("Invert Depth", &invertDepth);
    ImGui::SliderFloat("Depth Multiplier", &depthMultiplier, 0.0f, 5.0f);

    // Status
    ImGui::SeparatorText("Status");
    ImGui::Text("Current color: %s", colorImagePath.c_str());
    ImGui::Text("Current depth: %s", depthImagePath.c_str());
    ImGui::Text("Image size: %dx%d", iw, ih);
    ImGui::Text("FPS: %.1f", io.Framerate);

    ImGui::End();
}

```

Az egész ciklust addig fogjuk futtatni, amíg ki nem lépünk a programból. Először is létrehozuk a UI-t. Lehet képeket betölteni, a teszt mintát használni, valamint a homályosítás értékét növelni / csökkenteni, valamint mutatjuk a kép méretét, a homályosítás mennyiségét és a jelenlegi FPS-t is

```

// Handle file dialogs
if (loadColorRequested) {
    loadColorRequested = false;
    std::string path = OpenFileDialog();
    if (!path.empty()) {
        strncpy(colorPathBuffer, path.c_str(), sizeof(colorPathBuffer) - 1);
        colorPathBuffer[sizeof(colorPathBuffer) - 1] = '\0';
    }
}

if (loadDepthRequested) {
    loadDepthRequested = false;
    std::string path = OpenFileDialog();
    if (!path.empty()) {
        strncpy(depthPathBuffer, path.c_str(), sizeof(depthPathBuffer) - 1);
        depthPathBuffer[sizeof(depthPathBuffer) - 1] = '\0';
    }
}

// Handle save output request
if (saveOutputRequested) {
    saveOutputRequested = false;

    clFinish(queue);

    std::vector<float> floatPixels = ReadTexturePixelsFloat(outputTexture, w, h);

    // Convert to 8-bit
    std::vector<unsigned char> bytePixels(w * h * 4);
    for (size_t i = 0; i < floatPixels.size(); i++) {
        float val = clamp(floatPixels[i] * 255.0f, 0.0f, 255.0f);
        bytePixels[i] = static_cast<unsigned char>(val);
    }

    // Open save dialog
    std::string savePath = SaveFileDialog();
    if (!savePath.empty()) {
        SaveImage(savePath, bytePixels, w, h, 4);
    }
}

// Handle reload requests
if (reloadRequested) {
    reloadRequested = false;
    if (!loadImages()) {
        std::cout << "Failed to load images, using test pattern instead" << std::endl;
        createTestPattern();
    }
}

if (useTestPatternRequested) {
    useTestPatternRequested = false;
    createTestPattern();
}

```

Képek tényleges betöltése és a mentés is itt történik, valamint, ha akarjuk cserélni a jelenlegi képeinket, akkor tudjuk és akár vissza is térhetünk a teszteléshez használt sakktábla szerű képhez is

A kettő képnek kötelező **520 x 520**-nak lennie, különben a program nem fog lefutni!


```

// Clear screen
glClearColor(0.1f, 0.1f, 0.1f, 1.0f);
glClear(GL_COLOR_BUFFER_BIT);

// Acquire OpenGL objects for OpenCL
clEnqueueAcquireGLObjects(queue, 1, &outputBuffer, 0, nullptr, nullptr);
if (showDepthMap) {
    clEnqueueAcquireGLObjects(queue, 1, &depthVisBuffer, 0, nullptr, nullptr);
}

if (useSATMethod) {
    // Clear SAT buffers
    const cl_float4 zero4 = { 0.0f, 0.0f, 0.0f, 0.0f };
    clEnqueueFillBuffer(queue, satBuffer, &zero4, sizeof(zero4), 0, colorBufferSize, 0, nullptr, nullptr);
    clEnqueueFillBuffer(queue, tempBuffer, &zero4, sizeof(zero4), 0, colorBufferSize, 0, nullptr, nullptr);

    // Step 1: Horizontal prefix sum
    size_t global_h = h;
    clSetKernelArg(sat_horizontal, 0, sizeof(cl_mem), &colorBuffer);
    clSetKernelArg(sat_horizontal, 1, sizeof(cl_mem), &tempBuffer);
    clSetKernelArg(sat_horizontal, 2, sizeof(int), &w);
    clSetKernelArg(sat_horizontal, 3, sizeof(int), &h);
    clEnqueueNDRangeKernel(queue, sat_horizontal, 1, nullptr, &global_h, nullptr, 0, nullptr, nullptr);

    // Step 2: Vertical prefix sum
    size_t global_v = w;
    clSetKernelArg(sat_vertical, 0, sizeof(cl_mem), &tempBuffer);
    clSetKernelArg(sat_vertical, 1, sizeof(cl_mem), &satBuffer);
    clSetKernelArg(sat_vertical, 2, sizeof(int), &w);
    clSetKernelArg(sat_vertical, 3, sizeof(int), &h);
    clEnqueueNDRangeKernel(queue, sat_vertical, 1, nullptr, &global_v, nullptr, 0, nullptr, nullptr);

    // Step 3: Apply depth blur using SAT
    size_t global_blur[2] = { (size_t)w, (size_t)h };
    float adjustedBlurRadius = blurRadius * depthMultiplier;
    int invertDepthInt = invertDepth ? 1 : 0;

    clSetKernelArg(blurKernel, 0, sizeof(cl_mem), &colorBuffer);
    clSetKernelArg(blurKernel, 1, sizeof(cl_mem), &depthBuffer);
    clSetKernelArg(blurKernel, 2, sizeof(cl_mem), &satBuffer);
    clSetKernelArg(blurKernel, 3, sizeof(cl_mem), &outputBuffer);
    clSetKernelArg(blurKernel, 4, sizeof(int), &w);
    clSetKernelArg(blurKernel, 5, sizeof(int), &h);
    clSetKernelArg(blurKernel, 6, sizeof(float), &adjustedBlurRadius);
    clSetKernelArg(blurKernel, 7, sizeof(int), &invertDepthInt);
    clEnqueueNDRangeKernel(queue, blurKernel, 2, nullptr, global_blur, nullptr, 0, nullptr, nullptr);
}
else {
    // Use simple box blur
    size_t global_blur[2] = { (size_t)w, (size_t)h };
    float adjustedBlurRadius = blurRadius * depthMultiplier;
    int invertDepthInt = invertDepth ? 1 : 0;

    clSetKernelArg(simpleBlurKernel, 0, sizeof(cl_mem), &colorBuffer);
    clSetKernelArg(simpleBlurKernel, 1, sizeof(cl_mem), &depthBuffer);
    clSetKernelArg(simpleBlurKernel, 2, sizeof(cl_mem), &outputBuffer);
    clSetKernelArg(simpleBlurKernel, 3, sizeof(int), &w);
    clSetKernelArg(simpleBlurKernel, 4, sizeof(int), &h);
    clSetKernelArg(simpleBlurKernel, 5, sizeof(float), &adjustedBlurRadius);
    clSetKernelArg(simpleBlurKernel, 6, sizeof(int), &invertDepthInt);
    clEnqueueNDRangeKernel(queue, simpleBlurKernel, 2, nullptr, global_blur, nullptr, 0, nullptr, nullptr);
}

```

Itt történik az igazi homályosítás, először kiürítjük az SAT Buffert, hogy ne maradjanak meg az előző kép adatai, majd összeadjuk a vízszintes prefixek eredményét, majd a függőleges eredményeket a kerneleket használva, végül az SAT-re alkalmazzuk a homályosítást

Ha nem használunk SAT-t, akkor meg csak átadjuk a szimpla boks blur kernelnek a képek adatait, de SAT használatával gyorsabb lesz, mint a boks blur használatával

Valamint meglehet nézni az eredeti depth képet, referencia kedvéből


```

// Visualize depth map if requested
if (showDepthMap) {
    size_t global_depth[2] = { (size_t)w, (size_t)h };
    int invertDepthInt = invertDepth ? 1 : 0;

    clSetKernelArg(visualizeDepthKernel, 0, sizeof(cl_mem), &depthBuffer);
    clSetKernelArg(visualizeDepthKernel, 1, sizeof(cl_mem), &depthVisBuffer);
    clSetKernelArg(visualizeDepthKernel, 2, sizeof(int), &w);
    clSetKernelArg(visualizeDepthKernel, 3, sizeof(int), &h);
    clSetKernelArg(visualizeDepthKernel, 4, sizeof(int), &invertDepthInt);
    clEnqueueNDRangeKernel(queue, visualizeDepthKernel, 2, nullptr, global_depth, nullptr, 0, nullptr, nullptr);
}

// Release OpenGL objects
clEnqueueReleaseGLObjets(queue, 1, &outputBuffer, 0, nullptr, nullptr);
if (showDepthMap) {
    clEnqueueReleaseGLObjets(queue, 1, &depthVisBuffer, 0, nullptr, nullptr);
}

clFinish(queue);

// Render output
ImGui::Begin("Output");
ImGui::Image((void*)(intptr_t)outputTexture, ImVec2(w, h));
ImGui::End();

if (showDepthMap) {
    ImGui::Begin("Depth Map");
    ImGui::Image((void*)(intptr_t)depthTexture, ImVec2(w, h));
    ImGui::End();
}

// Render ImGui
ImGui::Render();
ImGui_ImplOpenGL3_RenderDrawData(ImGui::GetDrawData());

// Swap buffers
glfwSwapBuffers(window);

```

Itt a depth képet megjelenítjük egy új ablakban

```

// Cleanup
if (imgData) stbi_image_free(imgData);
if (depData) stbi_image_free(depData);

clReleaseMemObject(colorBuffer);
clReleaseMemObject(depthBuffer);
clReleaseMemObject(satBuffer);
clReleaseMemObject(outputBuffer);
clReleaseMemObject(depthVisBuffer);

clReleaseKernel(sat_horizontal);
clReleaseKernel(sat_vertical);
clReleaseKernel(blurKernel);
clReleaseKernel(simpleBlurKernel);
clReleaseKernel(visualizeDepthKernel);
clReleaseProgram(program);
clReleaseCommandQueue(queue);
clReleaseContext(context);
clReleaseMemObject(tempBuffer);

glDeleteTextures(1, &outputTexture);
glDeleteTextures(1, &depthTexture);

ImGui_ImplOpenGL3_Shutdown();
ImGui_ImplGlfw_Shutdown();
ImGui::DestroyContext();

glfwDestroyWindow(window);
glfwTerminate();

return 0;

```

Végül a takarítás, minden Kernelt releaselünk, a képeket szintűgy és a buffereket is, az output és a depth textúrákat töröljük, és kikapcsoljuk az ImGui és a GLFW működését